

CSE 599k: LLM Serving Systems – Assignment 1

In this assignment, you will practice GPU programming using **PyTorch**, **Triton**, and **CUDA**, along with **profiling techniques**. The assignment consists of three parts: **SiLU**, **RMS Norm**, and **Host-GPU Communication**. Your submission will be in the form of a .zip file upload on Canvas, and must include a **short report with answers to the below questions (.pdf)**, **code**, and **profiling results**. For performance-related questions, you will receive **0 points** if your **kernel is incorrect**. If your kernel is correct, your score will be determined **linearly based on its performance**. You will receive **full grade** if your kernel meets or exceeds the **performance standard, which is around 60-70% performance of our reference kernels**.

Section 1: Implementing the Sigmoid Linear Unit (SiLU)

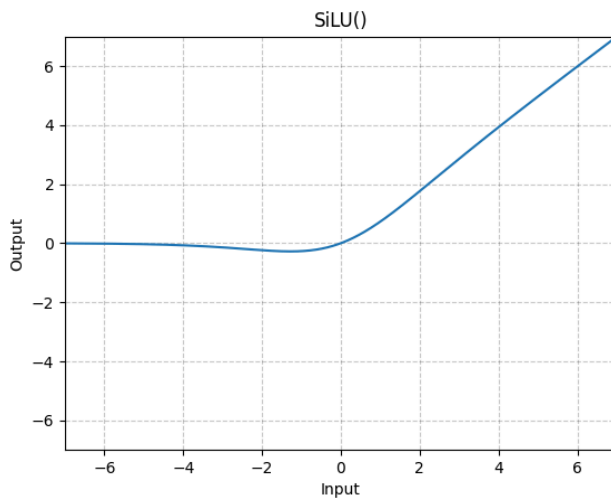


Image source: <https://pytorch.org/docs/stable/generated/torch.nn.SiLU.html>

Q1. 1 + 1 points

- What are the use cases of SiLU in LLMs?

SiLU is used as an activation function in LLMs usually in the MLP layers but can also be used in the transformer block. Generally it can be considered a good alternative to the ReLU function as it avoids the “dead neuron problem” that ReLU encounters since it outputs exactly 0 if the input is ≤ 0 .

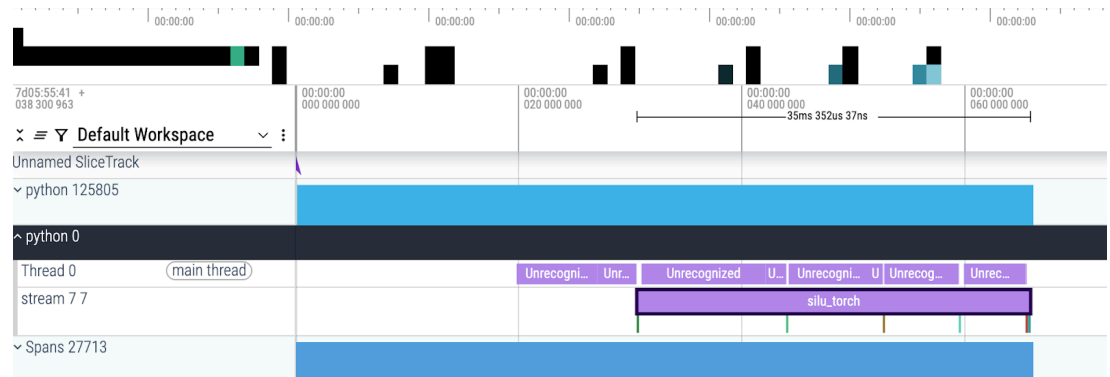
- What is the mathematical formula for the SiLU function?

$$\frac{x}{1 + e^{-x}}$$

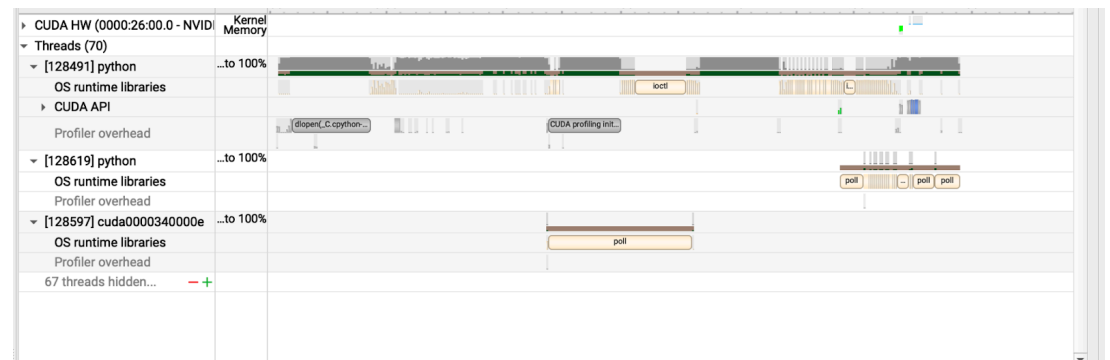
Q2. 2 + 4 + 2 points

- In PyTorch, without using `nn.SiLU` or `nn.Sigmoid`, implement the SiLU function using basic tensor operations for a matrix of shape `(8192, 8192)` in `silu/silu_torch.py`.
- Use **Torch Profiler** and **NVIDIA Nsight Systems** to profile the execution.
 - Include **screenshots** in the report.Submit the profiling outputs as:

- `torch_silu.json` (Torch Profiler)



- `torch_silu.nsys-rep` (Nsight Systems)



- Analyze and answer the following:
 - What kernels are being launched?

When we profiled the SiLU function with `nsys`, we saw the following CUDA kernels being launched:

- `void at::native::vectorized_elementwise_kernel<(int)4, at::native::BinaryFunctor<float, float, float, at::native::binary_internal::MulFunctor<float>>, std::array<char *, (unsigned long)3>>(int, T2, T3)`
- `void at::native::vectorized_elementwise_kernel<(int)4, at::native::exp_kernel_cuda(at::TensorIteratorBase`

```
&)::[lambda() (instance 2)]::operator ()()
const::[lambda() (instance 2)]::operator ()()
const::[lambda(float) (instance 1)], std::array<char
*, (unsigned long)2>>(int, T2, T3)
```

- void at::native::vectorized_elementwise_kernel<(int)4, at::native::CUDAFunctorOnSelf_add<float>, std::array<char *, (unsigned long)2>>(int, T2, T3)
- void at::native::vectorized_elementwise_kernel<(int)4, at::native::AUnaryFunctor<float, float, float, at::native::binary_internal::MulFunctor<float>>, std::array<char *, (unsigned long)2>>(int, T2, T3)
- void at::native::vectorized_elementwise_kernel<(int)4, at::native::reciprocal_kernel_cuda(at::TensorIteratorBase &)::[lambda() (instance 1)]::operator ()() const::[lambda() (instance 2)]::operator ()() const::[lambda(float) (instance 1)], std::array<char *, (unsigned long)2>>(int, T2, T3)
- void at::native::vectorized_elementwise_kernel<(int)4, at::native::neg_kernel_cuda(at::TensorIteratorBase &)::[lambda() (instance 2)]::operator ()() const::[lambda() (instance 7)]::operator ()() const::[lambda(float) (instance 1)], std::array<char *, (unsigned long)2>>(int, T2, T3)

These match up with the high-level operations which are shown in the Torch Profiler results: `aten::neg`, `aten::exp`, `aten::add`, `aten::reciprocal`, and `aten::mul`.

The specific order these show up in the timeline is:

```
aten::neg, aten::exp, aten::add, aten::reciprocal, aten::mul,
aten::mul
```

- What is the minimum number (discount repetition and intermediate results) of memory accesses necessary (read and write) to accomplish the task?

If we only count the absolute minimum necessary memory accesses and ignore intermediate results being written to memory, it's just the input and the final output. That means **one read of x and one write of the final output**. Since the tensor is shape (8192, 8192), that's 67,108,864 elements. So total accesses are $2 \times 67,108,864 = 134,217,728$ which is **512 MiB assuming float32**. Everything in between like the negation, exponential, addition, and reciprocal can all be computed in registers without extra memory accesses if the kernel is fully optimized.

- What is the bandwidth usage (minimum number of necessary memory accesses / total execution time) of SiLU?

Since the total execution time of the `silu_torch` function is 59ms 706us 557ns or **59.706557 ms** we find that the bandwidth in MiB/ms is **512 MiB/59.706557 ms = 8.575 MiB/ms = 8.990 GB/s**.

Q3. 8 + 2 points

- Implement the SiLU function using **Triton** in `silu/silu_triton_kernel.py` and test your kernel in `silu/silu_triton_test.py`.
- Time the custom Triton kernel and report the timing and bandwidth.

Triton kernel time: 0.00018211103439331057 s

Memory Access: $8192 * 8192 * 2 * 4 = 536,870,912$ bytes = 0.536870912 GB

Bandwidth: $0.536870912 / 0.00018211103439331057 = \mathbf{2948.042 \text{ GB/s}}$

Q4. 8 + 2 + 10 points

- Implement the SiLU function using **CUDA** in `silu/CUDA/silu.cu`. Test your kernel in `silu/CUDA/main.cu`.
- Time the kernel and compute the bandwidth utilization. Use **NVIDIA Nsight Compute (ncu)** to analyze your CUDA kernel. Include the report `cuda_silu.ncu-rep` in your submission.
- **Optimize** the kernel to achieve **bandwidth utilization > 2000 GB/s**.

Total time: 0.262496 ms = 0.000262496 s

Memory Access: $8192 * 8192 * 2 * 4 = 536,870,912$ bytes = 0.536870912 GB

Bandwidth: $0.536870912 / 0.000262496 = \mathbf{2045.254 \text{ GB/s}}$

Section 2: Implementing RMS Norm

Q1. 1 + 1 + 1 points

- What is the RMS Norm?
 - A layer normalization technique to rescale inputs according to its root mean square, particularly foregoing rescaling of the mean to save computation
- What is RMSNorm used for in LLMs?
 - It is used to improve stability of computations on data similar to other layer normalization techniques. For instance, it is used before MLP layers to make gradients more stable so learning is more efficient.
- Provide the formula for RMS Norm.

$$RMS = \sqrt{\epsilon + \frac{1}{n} \sum_{i=1}^n x_i^2} \quad x_i = \frac{x_i}{RMS(x)} g_i$$

[RMSNorm paper](#)

Q2. 10 + 10 points

- Implement a **CUDA kernel** for RMS Norm for a matrix of shape (8192, 8192) in `rms_norm/matrix/rms_norm_matrix.cu`, write a test in `rms_norm/matrix/main.cu`. Time the kernel and compute the bandwidth utilization
- Optimize it to achieve **bandwidth utilization > 1000 GB/s** (attach `rms_norm_matrix.ncu-rep`).

Total Time: 0.357035 ms = 0.000357035 s

Memory Access: $8192 * 8192 * 3 * 4 + 8192 * 4 = 805339136$ bytes = 0.805339136 GB

- This is 2 reads of the input, one write of the output and one read of weights with everything else being in shared memory.

Bandwidth: $0.805339136 / 0.000357035 = 2255.63$ GB/s



Q3. 10 + 10 points

- Implement a **CUDA kernel** for RMS Norm for a vector of shape $(1, 1024 \times 1024)$ in `rms_norm/vector/rms_norm_vector.cu`, write a test in `rms_norm/vector/main.cu`. Time the kernel and compute the bandwidth utilization
- Optimize this kernel as well to achieve **bandwidth utilization > 400 GB/s** (attach `rms_norm_vector.ncu-rep`).

Total time: 0.025043 ms = 0.000025043s

Memory Access: $1 * (1024 * 1024) * 4 * 4 = 16777216$ bytes = 0.016777216 GB

- This is two reads of input, one read of weight, and one write of output since everything else happens inside of shared memory.

Bandwidth: $0.016777216 / 0.000025043 = 669.936$ GB/s

Q4. 2 points

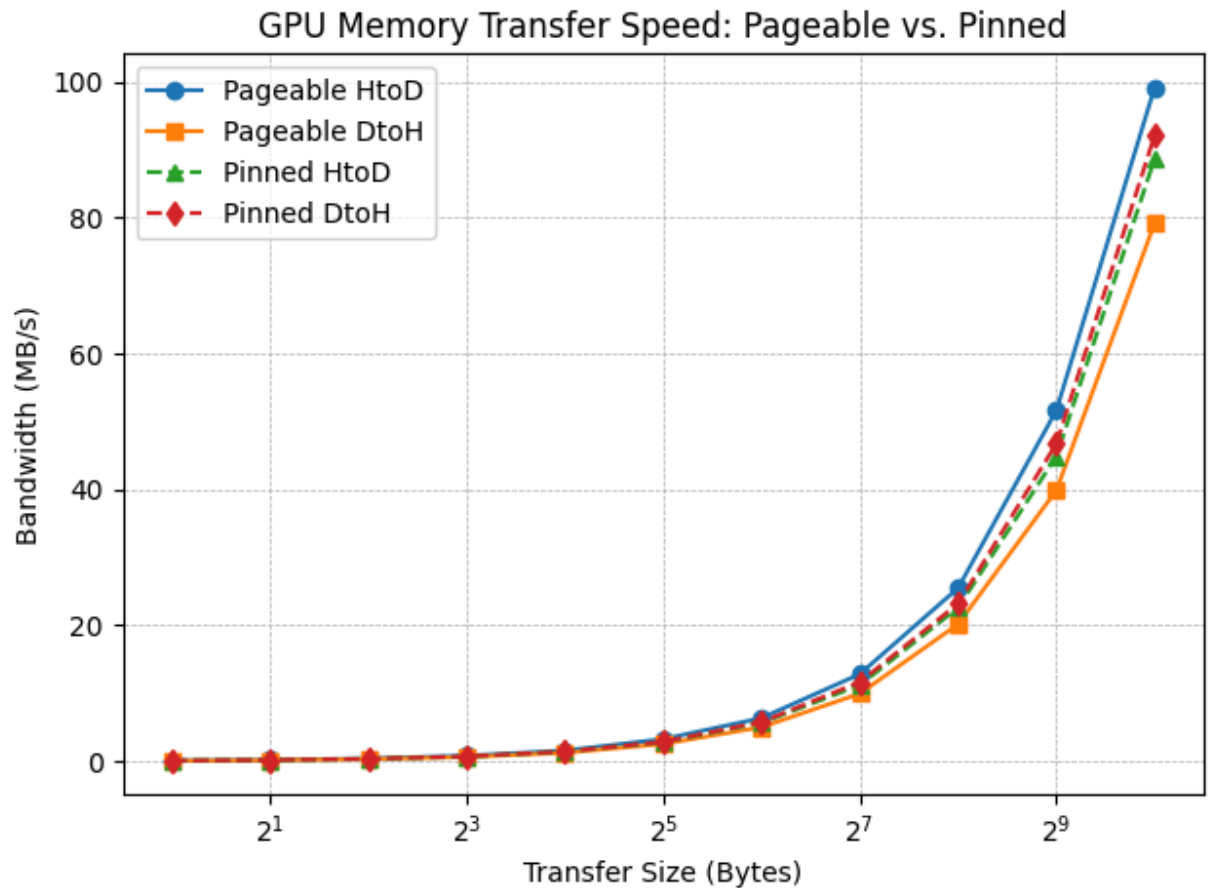
- Compare the thread block assignment for kernels implemented for $(8192, 8192)$ and $(1, 1024 \times 1024)$. What are the thread block assignment differences, and why were these changes necessary?

The main difference between the two kernels is that the matrix kernel uses row-wise parallelism, assigning one thread block per row so that all threads in a block collaborate to normalize that row, while the vector kernel uses element-wise parallelism, assigning threads across the entire vector with all the threads working together to do a reduction and then each thread responsible for normalizing one output. This change in thread block assignment was necessary because the matrix version needs to handle multiple independent rows in parallel, whereas the vector version is just a single row of data, so spreading threads across elements directly is more efficient. The matrix kernel exploits row-level independence, while the vector kernel focuses on parallelizing over individual elements.

Section 3: Host-GPU Memory Copy

Q1. 2 + 2 + 1 points

- Measure the memory transfer speed:
 - **Host-to-GPU** and **GPU-to-Host**
For data sizes of 2^0 to 2^{10} bytes for each power of 2.
 - Use **C++**
- Plot a **bandwidth vs. transfer size** curve and include it in your report.



- What is the **peak transfer speed** observed?

Peak Pageable Host-to-Device Bandwidth: 99.15 MB/s

Peak Pageable Device-to-Host Bandwidth: 79.18 MB/s

Peak Pinned Host-to-Device Bandwidth: 88.67 MB/s

Peak Pinned Device-to-Host Bandwidth: 92.11 MB/s

Q2. 2 + 2 points

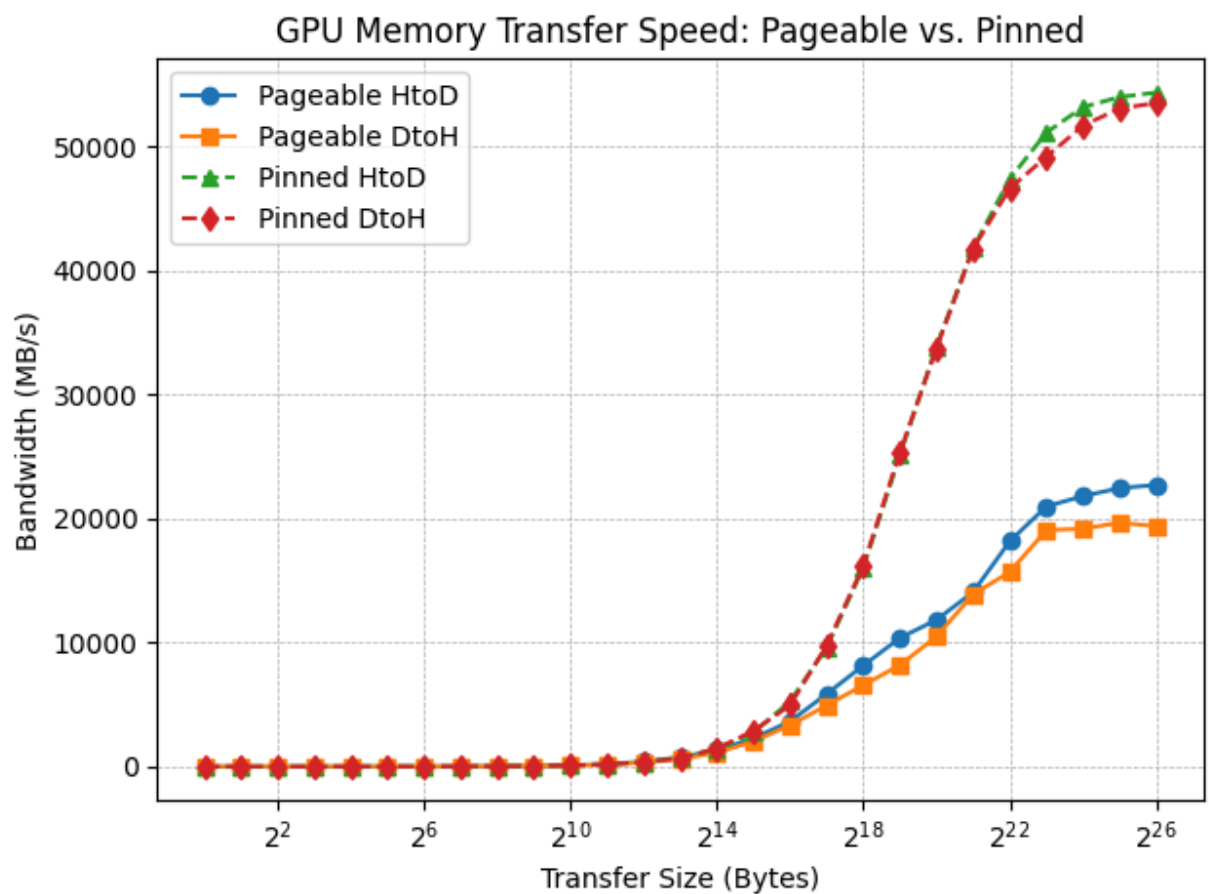
- What is **pinned memory** in the context of NVIDIA GPUs' memory allocations?

Pinned memory is a type of host memory that the OS can't page out of memory (to disk) which makes memory transfers between the CPU and GPU (potentially) faster.

- How does it impact the performance of memory transfers between the CPU and GPU?
Add a pinned memory curve in the figure for Q1 (Section 3).

For small data transfers (where paging won't even happen due to data size) there will be little to no difference because it would have been transferred with minimal (e.g. 1) page from disk (see plot in section 3 question 1 and peak transfer speeds). However when we get to larger data which can potentially be paged out of memory we start to see very significant (~2x) bandwidth improvements for transfer (see plot below)

When using paged (non-pinned) memory, an extra copy is made to a staging area before the transfer, which would explain the roughly 2x speedup we see, since we're doing half the copies.



Q3. 3 + 3 points

- Assume a matrix of shape (8192, 65536) is on the **CPU** in [row-major](#) order. Implement an efficient **C++ host-to-GPU communication** routine in [host_GPU/copy_first_column.cu](#) to:
 - Copy **only the first column** of each row to the GPU.

- **Optimize** the transfer to ensure the total time is **under 100 μ s (include copy.nsys-rep)**.